

MATERIAL DE ESTUDIO

NLP Explorer

Fundamentos de NLP moderno:
Tokenización avanzada y embeddings
semánticos



Introducción · ¿Cómo puede un ordenador trabajar con lenguaje?

¿Cómo puede un ordenador trabajar con lenguaje si, en último término, trabaja con números?

Podemos escribir «Quiero cambiar mi contraseña» y reconocer que «Necesito modificar mi clave de acceso» expresa una intención muy parecida. Apenas necesitamos detenernos para hacerlo. Para un programa, sin embargo, son dos secuencias de caracteres distintas. ¿Qué tendría que ocurrir para relacionarlas sin exigir que contengan las mismas palabras?

Esta es una de las preguntas del procesamiento del lenguaje natural, abreviado **PLN** en español y **NLP** por su nombre en inglés, *Natural Language Processing*. Este campo desarrolla métodos para trabajar computacionalmente con lenguaje: buscar información, organizar mensajes, clasificar textos o generar respuestas, entre otras tareas. Aquí nos concentraremos en cómo representar texto y comparar esas representaciones.

Dos piezas serán centrales: la **tokenización**, que prepara las unidades de texto que recibe el modelo, y los **embeddings semánticos**, representaciones numéricas aprendidas que pueden reflejar relaciones entre usos del lenguaje. Para entender por qué importan, recorreremos primero varios intentos más sencillos. Cada uno resuelve un problema y deja al descubierto otro. Ese recorrido es una guía conceptual, no una historia en la que toda técnica nueva vuelve inútiles las anteriores.

No hace falta memorizar fórmulas ni conocer la arquitectura interna de una red neuronal. El objetivo es comprender qué aporta cada transformación y poder interpretar un resultado sin atribuirle más de lo que demuestra. Todos los ejemplos se explican aquí; no necesitas ejecutar código, abrir una aplicación ni haber asistido a una clase.

Guía de lectura

El recorrido comienza con identificadores y conteos, pasa al aprendizaje de relaciones y contexto, y vuelve después a la entrada del modelo para aclarar qué son los tokens. Con esas piezas unidas, compararemos frases y examinaremos aplicaciones, límites y decisiones de uso. Al final encontrarás un reto con interpretación separada, un glosario y preguntas de autoevaluación con respuestas.

Índice del recorrido

- [Secciones 1-4: identificadores, one-hot, Bag of Words y TF-IDF.](#)
- [Secciones 5-8: embeddings, Word2Vec, contexto, atención y Transformers.](#)
- [Secciones 9-13: tokenización, subwords, algoritmos y pipeline completo.](#)
- [Secciones 14-15: embeddings de frase y similitud del coseno.](#)
- [Secciones 16-20: aplicaciones, limitaciones, sesgos, privacidad y recursos.](#)
- [Secciones 21-22: reto de repaso y cierre.](#)
- [Consulta y repaso: glosario y autoevaluación con respuestas.](#)

1. Dar un número a una palabra todavía no representa su significado

Un ordenador puede guardar texto mediante códigos numéricos. Pero almacenar las letras de una palabra no resuelve cómo relacionarla con otra. Para empezar con algo manejable, imaginemos un pequeño catálogo:

```
perro = 1  
gato  = 2  
avión = 3
```

Ahora podemos referirnos a cada palabra sin escribirla entera. El número funciona como un identificador, o **ID**: permite encontrar una entrada concreta del catálogo. Es parecido al número de un expediente; distingue uno de otro, pero no explica su contenido.

¿Gato es el doble de perro? ¿Avión está semánticamente más lejos porque su ID es mayor? No. Asignamos esos números por convenio. Podríamos intercambiarlos sin cambiar nada sobre perros, gatos o aviones. Restar los IDs o comparar sus tamaños no nos daría una medida de significado.

La distinción que necesitamos conservar es **identificador frente a representación**. Un identificador responde «¿qué entrada es?». Una representación útil para una tarea debe permitir trabajar con alguna propiedad o relación relevante. Si queremos comparar significado, necesitamos que la relación entre números tenga una justificación distinta del orden de un catálogo.

2. Una casilla para cada palabra: one-hot

Podemos evitar ese falso orden reservando una casilla independiente para cada palabra. Fijamos las posiciones en el orden perro, gato, avión, y escribimos un uno en la casilla correspondiente y ceros en las demás:

```
      perro  gato  avión
perro → [ 1,   0,   0 ]
gato  → [ 0,   1,   0 ]
avión → [ 0,   0,   1 ]
```

Esta representación se llama **one-hot**: una sola posición está activa. Cada fila es un **vector**, es decir, una lista ordenada de números. Aquí el orden importa porque sabemos qué palabra corresponde a cada posición. El vector de gato no vale «el doble» que el de perro: simplemente activa otra casilla.

Hemos resuelto una confusión, pero todavía no hemos representado relaciones semánticas. Perro y gato no comparten ninguna posición activa; perro y avión tampoco. Con este esquema, las tres palabras son igual de independientes. Nada indica que las dos primeras designan animales y pueden compartir muchos contextos de uso.

Además, si nuestro **vocabulario**, el repertorio de entradas que admitimos, crece mucho, cada vector necesita muchas posiciones. Casi todas contendrán cero. A eso se le llama una representación **dispersa**: la información distinta de cero ocupa solo una pequeña parte de las casillas. Hay formas eficientes de almacenarla, pero su estructura sigue reservando una dimensión por entrada del vocabulario.

Catálogo didáctico · sin orden semántico

Palabra	ID	One-hot: perro gato avión
perro	1	[1, 0, 0]
gato	2	[0, 1, 0]
avión	3	[0, 0, 1]

Figura 1 · Identificar mediante un ID y mediante una posición one-hot.

Los IDs identifican entradas. One-hot activa una casilla por palabra, pero no expresa que perro y gato estén más relacionados que perro y avión

Una casilla por palabra puede parecer limitada para representar significado, pero nos permite abordar otra necesidad: describir textos completos mediante las palabras que contienen.

3. De palabras a documentos: Bag of Words

Supongamos que tenemos tres mensajes muy cortos: «perro gato», «perro avión» y «perro perro gato». En vez de elegir una sola casilla, podemos contar cuántas veces aparece cada palabra. Conservando el vocabulario anterior:

Texto	Conteos en el orden perro, gato, avión
perro gato	[1, 1, 0]
perro avión	[1, 0, 1]
perro perro gato	[2, 1, 0]

El último vector dice que perro aparece dos veces, gato una y avión ninguna. Hemos pasado de representar una palabra a representar un documento mediante un **vector de conteos**. «Documento» puede ser aquí un mensaje de unas pocas palabras; no tiene por qué ser un archivo largo.

Este método se conoce como **Bag of Words**, o bolsa de palabras, a menudo abreviado **BoW**. La metáfora de la bolsa señala su principal renuncia: conserva las palabras y sus frecuencias, pero no el orden en que aparecían.

Comparemos «perro persigue gato» con «gato persigue perro». Si las casillas son perro, persigue y gato, ambas frases producen [1, 1, 1]. Sin embargo, cambia quién persigue a quién. Los mismos ingredientes pueden contar historias diferentes, y la bolsa no conserva esa diferencia.

Conteos exactos del ejemplo

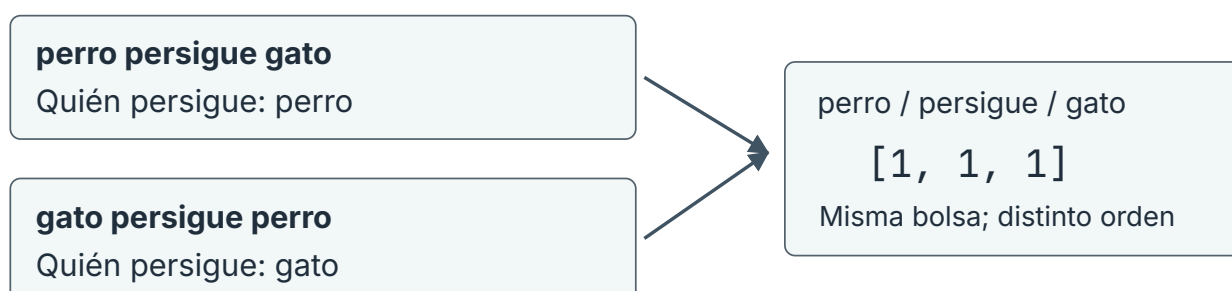


Figura 2 · Dos acciones distintas pueden producir la misma bolsa de palabras.

Las palabras y sus frecuencias coinciden. La bolsa de palabras no conserva quién realiza la acción

Esto no hace inútil a BoW. Para organizar textos por temas, la presencia de ciertas palabras puede ser una señal suficiente y fácil de revisar. Los conteos son interpretables y pueden resultar adecuados cuando importan la simplicidad y el coste. La pregunta es si la

información descartada resulta esencial para la tarea.

Aun aceptando perder el orden, surge otro problema: una palabra muy frecuente no siempre ayuda a distinguir un documento de los demás.

4. Cuando contar no basta: TF-IDF

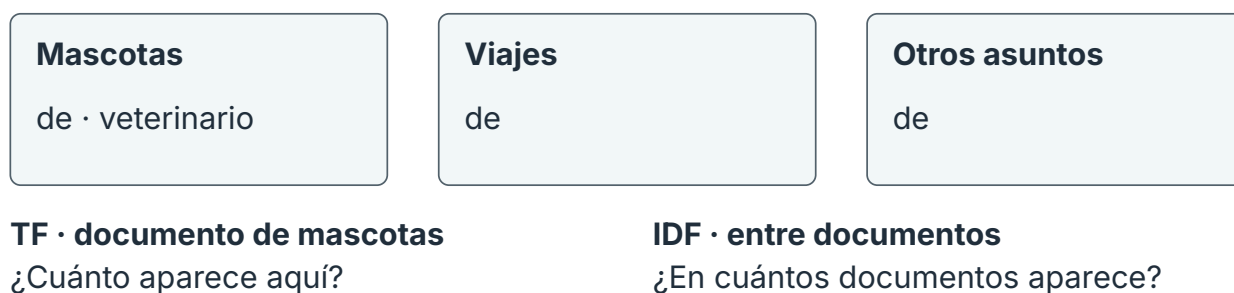
Imagina una colección de textos sobre animales, viajes y otros asuntos. Palabras como «de» aparecen una y otra vez en muchos de ellos. «Veterinario», en cambio, aparece en pocos y puede caracterizar mejor un texto sobre cuidados de mascotas. Si tratamos cada aparición como una señal igual de informativa, lo común puede tapar lo distintivo.

Necesitamos mirar en dos direcciones: dentro del documento y hacia el conjunto de documentos. **TF**, frecuencia del término, recoge cuánto aparece una palabra en el documento que estamos representando. **IDF**, frecuencia inversa de documento, ajusta su importancia según lo extendida que esté por la colección: aparecer en pocos documentos puede aportar más capacidad de distinguir que aparecer en casi todos.

TF-IDF combina ambas señales. Una palabra gana peso si tiene presencia en ese documento y resulta relativamente poco común en la colección. No hace falta calcular una fórmula para entender el cambio: ya no solo contamos, sino que ponderamos los conteos según cuánto ayudan a diferenciar textos.

Esta importancia depende de la colección. «Veterinario» puede distinguir un texto dentro de una biblioteca general y aportar menos si todos los documentos pertenecen a una clínica veterinaria. Tampoco toda palabra rara es valiosa: una errata puede ser muy poco frecuente sin ayudar a entender nada.

Colección esquemática · no es un cálculo de pesos



«Veterinario» distingue aquí; en otra colección puede distinguir menos.

Figura 3 · La presencia local y la rareza en la colección aportan señales distintas.

TF-IDF combina presencia en un documento con rareza en la colección. La importancia cambia si cambia la colección

TF-IDF mejora la comparación basada en palabras observadas, pero sigue apoyándose esencialmente en ellas. «Cambiar mi contraseña» y «modificar mi clave de acceso» pueden compartir una intención aunque utilicen términos distintos. La ponderación, por sí sola, no aprende que contraseña y clave de acceso pueden estar relacionadas. Para eso necesitamos otra clase de representación.

5. Del conteo a las relaciones aprendidas

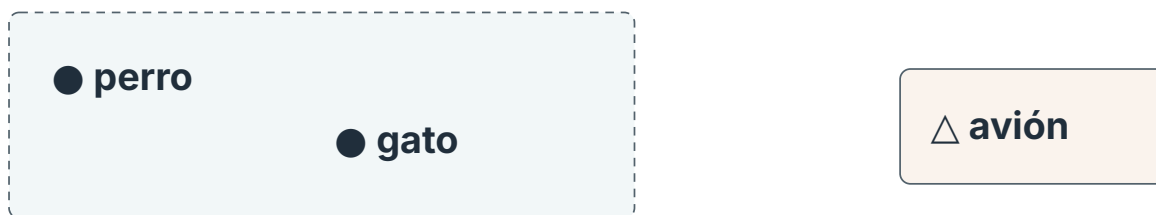
Volvamos a perro, gato y avión. Si la tarea trata sobre usos generales del lenguaje, esperamos que perro y gato compartan más relaciones entre sí que perro y avión. Pueden aparecer al hablar de mascotas, alimentación o cuidados. ¿Cómo conseguimos que esa información aparezca en sus representaciones?

Imaginemos un espacio en el que cada palabra ocupa una posición. Nos gustaría que ciertas relaciones se reflejaran en cercanías o direcciones, en lugar de tener casillas completamente independientes. Ya no asignaríamos posiciones solo para distinguir entradas: las aprenderíamos a partir de ejemplos.

Una representación aprendida de este tipo se llama **embedding**. Sigue siendo un vector, una lista ordenada de números, pero sus valores se ajustan durante un proceso de aprendizaje. En ese proceso, el modelo modifica sus parámetros —los valores internos que determinan sus cálculos— para realizar mejor un objetivo a partir de datos. Algunas relaciones útiles pueden quedar reflejadas en la geometría de los vectores.

La idea de un mapa ayuda a imaginarlo, con dos precauciones. Primero, un embedding no contiene «el significado verdadero» de una palabra. Refleja relaciones aprendidas según un modelo, unos datos y un objetivo. La cercanía que resulta útil para una tarea puede no ser la adecuada para otra. Segundo, un dibujo en dos dimensiones es una metáfora visual. Los modelos reales pueden utilizar cientos de dimensiones, y no podemos dar necesariamente a cada una una etiqueta como «animal», «tamaño» o «velocidad».

Mapa conceptual · sin ejes ni coordenadas



Relación esperada en este ejemplo; no son embeddings reales.

Figura 4 · Perro y gato próximos frente a avión: mapa conceptual, no medición.

Mapa conceptual, no resultado medido ni proyección de embeddings reales. La cercanía depende del modelo, los datos y el objetivo; las dimensiones reales no tienen necesariamente nombres interpretables

Aquí el avance no consiste simplemente en tener más números, sino en aprender cómo organizarlos para una finalidad. Falta responder de dónde podrían salir las señales para hacerlo.

6. Aprender observando usos: Word2Vec

Aunque nadie nos entregue un mapa de significados, disponemos de textos donde las palabras aparecen acompañadas de otras. Perro y gato pueden compartir vecinos como «comida», «cuidar» o «mascota». Esas coincidencias ofrecen pistas sobre sus usos. No prueban que dos palabras signifiquen lo mismo: también palabras opuestas pueden aparecer en contextos parecidos.

Word2Vec fue un paso histórico importante en el aprendizaje de vectores de palabras. Sus dos planteamientos conocidos permiten entender la idea sin entrar en fórmulas. En **CBOW**, se usa el contexto cercano para intentar predecir una palabra. En **Skip-gram**, se parte de una palabra para intentar predecir elementos de su contexto. Son dos maneras de convertir las regularidades de los textos en una tarea de aprendizaje. Véase el [trabajo sobre las representaciones de Word2Vec](#).

Imagina ocultar una palabra en «el gato come su comida». Las que quedan alrededor ofrecen información para intentar recuperarla. Si repetimos una tarea así sobre muchos ejemplos, el modelo puede ajustar vectores que resulten útiles para reconocer regularidades. No necesita recibir una definición de diccionario de cada palabra para aprender esas relaciones.

La consecuencia es una representación que puede aportar más que identidad o frecuencia. Pero los embeddings de palabras de este tipo tienen un límite importante: una vez aprendido el modelo, cada palabra tiene esencialmente un vector fijo. Se suele hablar de **embeddings estáticos**. El vector recoge señales de sus diferentes usos sin cambiar para cada frase concreta.

Eso nos lleva a una palabra para la que un único punto resulta especialmente incómodo: banco.

7. Banco no siempre quiere decir lo mismo

Lee estas dos frases:

Me senté en un banco del parque.

Fui al banco a retirar dinero.

Interpretamos la primera como un asiento y la segunda como una entidad financiera. No necesitamos que cambie la escritura de «banco»: usamos las pistas de alrededor. «Parque» orienta una lectura; «retirar» y «dinero» orientan otra. También contamos con conocimiento y experiencia que no están escritos por completo en la frase.

¿Debería «banco» tener exactamente la misma representación en ambos contextos? Un vector fijo puede reunir información sobre sus usos, pero no separa por sí mismo cuál corresponde a esta aparición concreta. Queremos que la representación cambie según la secuencia donde está la palabra.

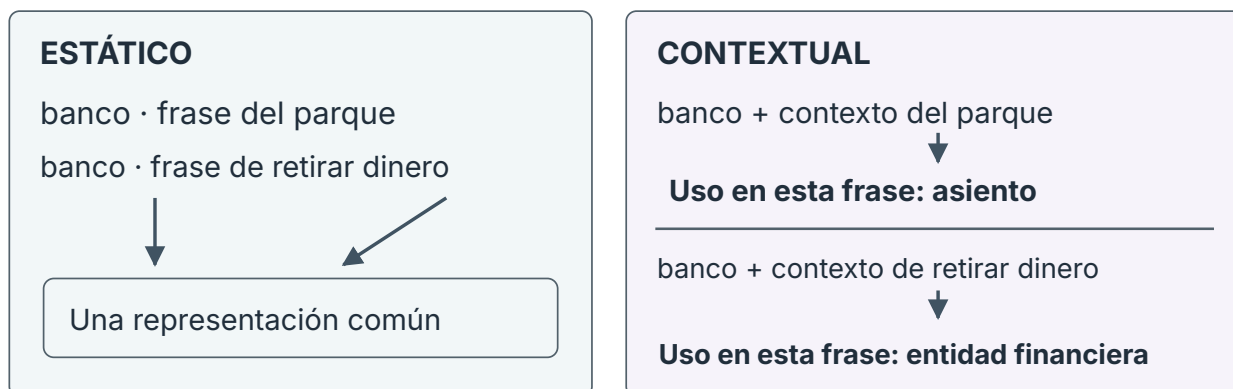
Esa es la idea de las **representaciones contextuales**: representar una aparición del texto teniendo en cuenta lo que la rodea. No implica crear un ID nuevo para cada sentido ni consultar necesariamente una lista de significados. Se trata de calcular una representación condicionada por el contexto disponible.

Contextos diferentes · mismo término «banco»

Me senté en un banco del parque.

Subrayado: pistas de contexto

Fui al banco a retirar dinero.



Esquema conceptual: no muestra atención ni vectores internos.

Figura 5 · Una representación estática frente a representaciones dependientes del contexto.

Las pistas del contexto pueden cambiar la representación de banco. El diagrama ilustra el mecanismo; no muestra vectores internos extraídos ni garantiza que todos los sentidos se distinguan correctamente

El contexto aporta señales, no certezas. «Nos vemos en el banco» puede seguir siendo ambiguo si no sabemos nada más. Para aprovechar las pistas que sí existen, necesitamos un mecanismo que permita combinar información de distintas partes de la secuencia.

8. Atención, Transformers y el ejemplo de BERT

Cuando interpretamos «banco» en la segunda frase, «retirar dinero» resulta más orientador que algunas de las otras palabras. Un modelo puede aprender a combinar información de distintas posiciones de la secuencia con pesos diferentes. Este mecanismo se llama **atención**. La palabra es una analogía útil; no supone que el sistema preste atención de forma consciente.

La idea central es que, al procesar una pieza, su representación puede incorporar información de otras. No necesita tratar cada pieza como una isla. Las relaciones que calcula dependen de lo aprendido y de la entrada concreta; un esquema que conecte banco con dinero ilustra esa posibilidad, pero no demuestra qué conexiones ha utilizado una ejecución real.

Un **Transformer** es una arquitectura, una forma de organizar el procesamiento del modelo, que hace central este tipo de intercambio de información. También necesita información sobre las posiciones de las piezas para que su orden cuente. No hace falta estudiar cada capa para entender el cambio: las representaciones se transforman teniendo en cuenta relaciones dentro de la secuencia.

BERT es un ejemplo conocido de Transformer de tipo **encoder**, o codificador: procesa una entrada y produce representaciones contextuales. Puede utilizar información a ambos lados de una posición, algo útil para distinguir usos como los de banco. Se presenta aquí como ejemplo de arquitectura contextual; no es el nombre del modelo de frases que utilizaremos después. El [artículo de BERT](#) desarrolla este planteamiento.

Contextual no significa infalible. Un modelo puede aprovechar mal una pista, heredar sesgos de sus datos o funcionar peor con textos distintos de los que conoce. Tampoco producir representaciones contextuales equivale automáticamente a obtener un buen vector para comparar frases enteras.

Antes de llegar a esa comparación, hay una pregunta que hemos dejado abierta: ¿qué entra realmente en el modelo? Hasta ahora hemos hablado de palabras por comodidad, pero las unidades que procesa no tienen por qué coincidir con ellas.

9. Preparar la entrada: texto, tokens e IDs

El modelo no recibe directamente «palabras con significado». Necesita una secuencia de unidades manejables. El proceso que prepara esas unidades se llama **tokenización**, y la herramienta que lo realiza, **tokenizador**. Cada unidad resultante es un **token**.

Un token puede coincidir con una palabra completa, ser parte de ella o representar puntuación. Su forma depende del tokenizador, de su vocabulario y de sus reglas. Por tanto, contar palabras separadas por espacios no siempre permite saber cuántos tokens tendrá un texto.

Observemos referencias reales del tokenizador asociado a sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2, el modelo que usaremos para representar frases:

```
perro    → ['_per', 'ro']
perros   → ['_per', 'ros']
perrito  → ['_per', 'rito']
perritos → ['_per', 'rito', 's']
```

Estas segmentaciones proceden de una ejecución previa de la práctica. Muestran las piezas de esas palabras sin añadir los tokens especiales que puede requerir el procesamiento completo. No son una división lingüística universal ni una propuesta de cómo deberíamos separar las palabras al escribir.

El símbolo `_` no es . En este tokenizador marca aproximadamente un inicio de palabra o un espacio previo; también puede aparecer al comienzo del texto. Forma parte de la manera de mostrar las piezas. Otros tokenizadores utilizan otras convenciones.

La pieza `_per` se reutiliza en las cuatro formas. «Perritos» no necesita una entrada independiente para toda la palabra: puede expresarse como una secuencia de piezas conocidas. A cada pieza le corresponde un ID dentro del vocabulario. Después, el modelo utilizará esos IDs para acceder a representaciones numéricas. Son pasos distintos:

```
texto → tokens → IDs → representaciones
```

Tokenizar prepara la entrada. Todavía no es el procesamiento contextual que permite interpretar el uso de una pieza dentro de una frase.

10. Por qué usar piezas de palabras

Podríamos reservar una entrada para cada palabra completa, pero tendríamos que incluir plurales, variaciones, nombres y muchas formas poco frecuentes. El vocabulario crecería y aun así aparecerían palabras que no figuraban en él. Un repertorio cerrado de palabras completas tiene dificultades para representar esas novedades.

El extremo opuesto sería separar todo en caracteres. El repertorio sería menor, pero muchas frases se convertirían en secuencias mucho más largas. El modelo necesitaría procesar más posiciones para trabajar con el mismo texto.

Las **subwords**, o piezas de palabras, ofrecen un compromiso: reutilizar fragmentos sin reducir necesariamente cada palabra a letras aisladas. Una subword puede ser una palabra completa o solo una parte. Es una unidad útil para el sistema, no una garantía de análisis lingüístico correcto. No tiene por qué coincidir con un morfema, una unidad con función o significado lingüístico como ciertos prefijos y sufijos.

El ejemplo de perritos lo hace visible: la división real es `_per`, `rito`, `s`. No debemos atribuir a cada fragmento un significado independiente. Del mismo tokenizador procede esta referencia adicional:

```
antidesestabilización → ['_anti', 'des', 'e', 'stabil', 'ización']
```

Segmentaciones observadas · sin tokens especiales

perro → ['_per', 'ro']

perritos → ['_per', 'rito', 's']

_ ≠ _ · marca aproximada de inicio de palabra / espacio previo

Piezas de este tokenizador; no son un análisis morfológico universal.

Figura 6 · Piezas reutilizadas en perro y perritos.

Segmentaciones observadas con el tokenizador asociado a sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2, sin tokens especiales. Las piezas dependen del tokenizador y no tienen por qué coincidir con morfemas

Poder dividir una palabra nueva en piezas conocidas ayuda a admitirla como entrada. No garantiza que el modelo sepa interpretar bien esa palabra. Además, el reparto entre vocabulario y longitud depende del diseño del tokenizador y del idioma. Ahora falta explicar cómo se construye ese repertorio de piezas.

11. Aprender un vocabulario y aplicarlo son cosas distintas

Una posibilidad consiste en partir de unidades pequeñas y buscar pares vecinos que se repitan mucho en una colección de textos, también llamada **corpus**. Se unen esos pares, se actualizan los conteos y se repite el proceso hasta alcanzar el tamaño previsto. Esa es la intuición de **BPE**, abreviatura de *Byte Pair Encoding*.

Como ejemplo hipotético, si p y e fueran el par vecino más frecuente, podríamos unirlos en pe; si después predominara pe seguido de r, podríamos formar per. Es una ilustración del procedimiento, no una reconstrucción del aprendizaje del tokenizador anterior. Las unidades iniciales y otros detalles varían según la implementación.

Primero se **aprende** un vocabulario y unas reglas a partir de una colección. Después, al **aplicar** el tokenizador, se utilizan las reglas ya aprendidas sobre el texto nuevo. BPE no vuelve a aprender sus fusiones desde cero cada vez que recibe una frase. La [documentación de BPE](#) explica esta construcción por fusiones.

BPE tampoco es sinónimo de tokenización en subwords. **WordPiece** construye piezas mediante un criterio de selección diferente: en lugar de basarse solo en el número de apariciones de un par, considera su contribución a explicar los datos. **Unigram** parte de

muchas piezas candidatas y va eliminando las que menos ayudan; utiliza probabilidades de las piezas para valorar segmentaciones. Son alternativas para construir y utilizar vocabularios, descritas en la [comparación de algoritmos de tokenización](#).

No necesitamos elegir un ganador universal entre ellas para comprender el recorrido. Lo esencial es que todas preparan unidades para el modelo. Ver las piezas de una palabra no basta para identificar con seguridad el algoritmo que las produjo, ni convertir esas piezas en IDs resuelve el significado.

12. Token $\neq$ ID $\neq$ embedding

Recuperemos banco, ahora desde la entrada del modelo. En las dos frases de la práctica se observó la misma pieza con el mismo identificador:

Elemento	Qué es
<code>_banco</code>	El token: la pieza de texto representada según la convención del tokenizador.
66799	Su ID: el índice que identifica esa entrada del vocabulario.
Un vector de números	Una representación numérica, por ejemplo un embedding inicial o una representación contextual posterior.

En la ejecución de referencia, `_banco` recibió el ID 66799 en ambas frases. Este valor corresponde al tokenizador utilizado en la práctica; no es una propiedad universal de la palabra «banco».

No hay nada en la magnitud 66799 que signifique «asiento», «dinero» o una mezcla de ambos. Ese número permite localizar una entrada en una tabla de vectores aprendidos. Consultar una tabla mediante un índice no es transformar matemáticamente el valor del índice en significado.

La misma pieza puede acceder al mismo vector de su tabla de entrada y, después, dar lugar a representaciones contextuales diferentes. Las posiciones y las demás piezas de la frase aportan información al procesamiento posterior. La identidad de vocabulario permanece; la representación de esa aparición puede cambiar.

Idea clave

El token es una pieza. El ID identifica esa pieza dentro de un vocabulario. El embedding es un vector. La contextualización procesa representaciones teniendo en cuenta la secuencia.

Observar que el ID se repite no equivale a medir cuánto cambian los vectores internos de banco. En la práctica de referencia se inspeccionaron tokens e IDs; la comparación entre sentidos mediante vectores internos se explica aquí de manera conceptual.

13. Unir las piezas: el recorrido completo

Ya podemos leer el proceso de principio a fin sin saltar de «palabra» a «significado» como si ocurriera una conversión directa:

```
Texto
↓
Tokens
↓
IDs
↓
Representaciones iniciales
↓
Transformer + contexto
↓
Representaciones contextualizadas
↓
Representación útil para una tarea
```

Del **texto a los tokens**, el tokenizador segmenta la entrada según sus reglas. De los **tokens a los IDs**, asigna a cada pieza el índice correspondiente en su vocabulario. El procesamiento completo puede añadir unidades especiales y otra información auxiliar; la lista de piezas visibles es una forma de inspeccionar la entrada, no necesariamente toda ella.

De los **IDs a las representaciones iniciales**, se consultan vectores aprendidos. El modelo incorpora además información de posición para tener en cuenta el orden. El tamaño del ID no determina el contenido del vector.

De las **representaciones iniciales al procesamiento del Transformer**, cada posición participa en cálculos que combinan información de la secuencia. El contexto pasa a influir en las representaciones resultantes. Las **representaciones contextualizadas** corresponden a posiciones de tokens; si una palabra se divide en varias piezas, no debemos confundir cada una con la palabra completa.

El último paso depende de lo que queramos hacer. Para detectar información en una parte de la frase pueden interesar sus tokens. Para comparar mensajes completos necesitamos una representación de frase. No son el mismo objeto ni tienen por qué obtenerse mediante un único procedimiento universal.

Pipeline conceptual · se muestra solo una pieza de la entrada

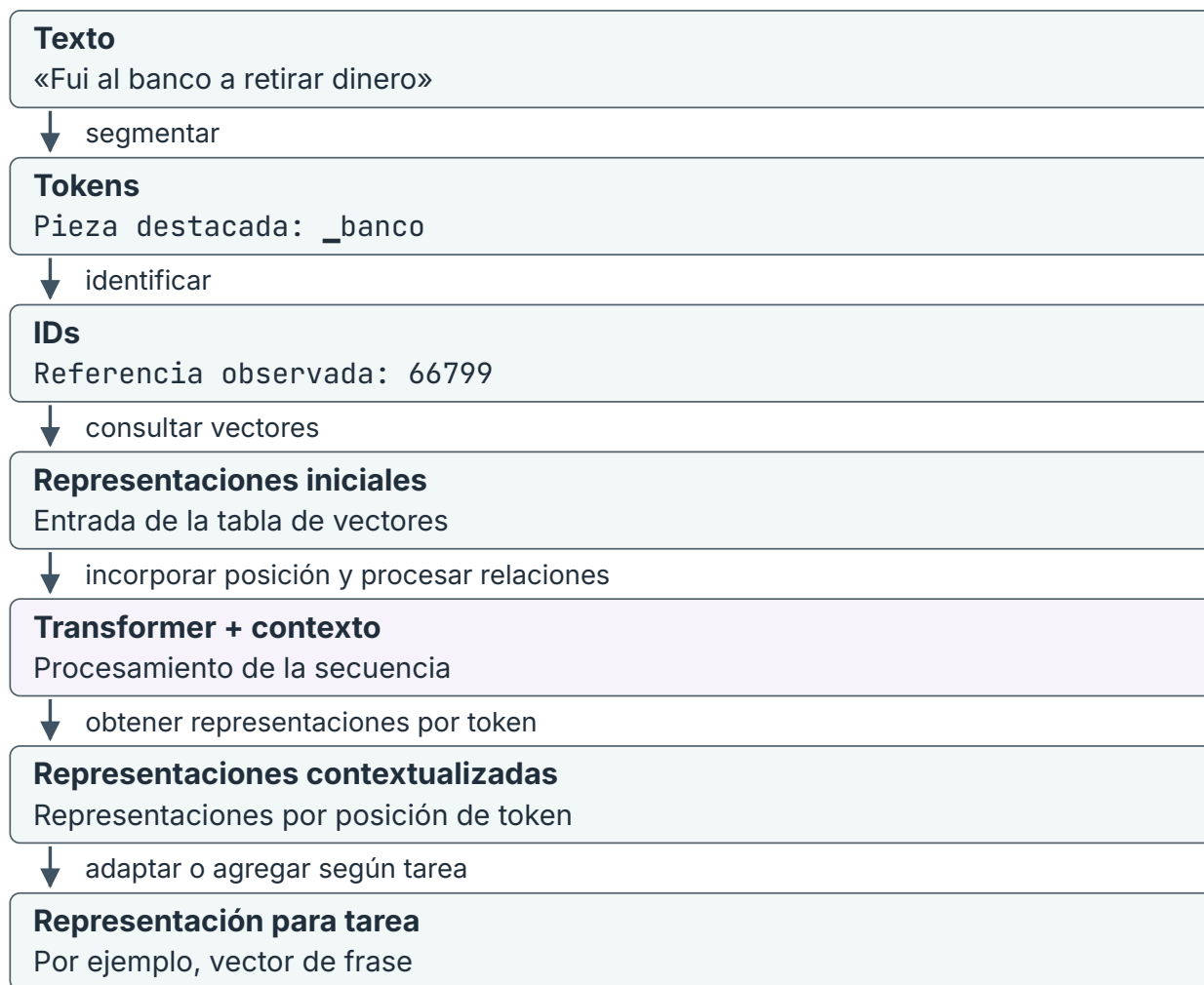


Figura 7 · De la preparación de la entrada al procesamiento contextual y la tarea.

El ID consulta una entrada; el procesamiento posterior incorpora contexto. El esquema omite detalles auxiliares. Solo la pieza destacada y su ID son referencias observadas; los vectores y relaciones se muestran de forma conceptual

Un **sentence embedding**, o embedding de frase, reúne información de una secuencia en un vector preparado para usos como la comparación. Algunos modelos combinan las representaciones de los tokens mediante una agregación, conocida como *pooling*; por ejemplo, pueden calcular un promedio teniendo en cuenta qué posiciones pertenecen al texto. Otros utilizan estrategias distintas. La manera de combinar y, especialmente, cómo se ha entrenado el sistema importan: promediar cualquier salida de cualquier Transformer no garantiza buenas comparaciones semánticas.

Para documentos largos surge una dificultad adicional. El modelo admite una longitud limitada de entrada y un solo vector comprime muchos detalles. Podemos representar fragmentos y compararlos por separado, o elegir un método preparado para documentos. La decisión

afecta a qué información conservamos: una condición importante escondida en un párrafo puede perderse al representar todo el documento de forma demasiado general.

14. Tres frases convertidas en vectores

Veamos una comparación concreta y completa:

```
A = "Quiero cambiar mi contraseña"  
B = "Necesito modificar mi clave de acceso"  
C = "Mañana va a llover"
```

A y B expresan peticiones relacionadas con cambiar una credencial de acceso. C habla del tiempo. Nuestra expectativa es que A y B tengan representaciones más próximas que A y C, aunque esa expectativa debe contrastarse con lo que produzca el modelo.

El modelo empleado es sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2, utilizado mediante la biblioteca Sentence Transformers. Es multilingüe, incluye el español y está preparado para obtener representaciones útiles de frases. Produce vectores de **384 valores**, tal como describe su [ficha pública](#).

En una ejecución previa de la práctica en CPU, con sentence-transformers==6.1.0, la forma observada fue:

```
torch.Size([3, 384])
```

Esta notación describe el tamaño de la estructura numérica resultante: **3 frases**, cada una representada por **384 valores**. Podemos imaginar tres filas con 384 números por fila. No son 384 palabras, ni 384 categorías con nombres, ni una puntuación global de calidad. Es la dimensión de la representación que entrega este modelo concreto.

No necesitamos leer e interpretar cada componente por separado. Lo que nos interesa es cómo se relacionan los vectores completos. Para que la comparación tenga sentido, A, B y C se representan con el mismo modelo y procedimiento; no basta con que dos modelos distintos entreguen vectores de igual longitud para poder mezclarlos.

Tenemos los números, pero todavía falta una regla para compararlos.

15. Comparar direcciones: similitud del coseno

Imagina cada vector como una flecha que parte del mismo origen. Dos flechas pueden tener longitudes diferentes y apuntar hacia una dirección parecida. Podemos comparar esa orientación, en lugar de fijarnos solo en cuánto mide cada flecha.

La **similitud del coseno** mide esa relación entre direcciones. Para vectores no nulos, una orientación idéntica da un valor de 1; direcciones perpendiculares, 0; direcciones opuestas, -1. Es una descripción geométrica. La interpretación semántica depende de que los vectores aprendidos organicen información útil para nuestra tarea.

Si quieres conectar la intuición con su expresión matemática:

$$\cos(A, B) = (A \cdot B) / (||A|| ||B||)$$

Aquí A y B representan los vectores, no las cadenas de texto originales. El **producto escalar** $A \cdot B$ se obtiene multiplicando los valores de la misma posición y sumando esos productos. La **norma** $||A||$ es la longitud de la flecha: se calcula como la raíz cuadrada de la suma de los cuadrados de sus componentes. Al dividir el producto escalar por las dos longitudes, se elimina el efecto de la escala y queda una cantidad relacionada con el ángulo entre ambas direcciones. No necesitas hacer esa operación a mano para interpretar el resultado.

Para las frases anteriores, las referencias observadas con el modelo indicado y cálculo de coseno fueron:

Comparación	Similitud observada aproximada
A ↔ B	0.7471
A ↔ C	-0.0577

A y B obtuvieron una similitud claramente mayor. Esto encaja con nuestra lectura: son dos formas de pedir un cambio de credencial. La comparación con la frase sobre lluvia queda cerca de cero. Ese pequeño valor negativo no significa que llover sea el contrario de cambiar una contraseña.

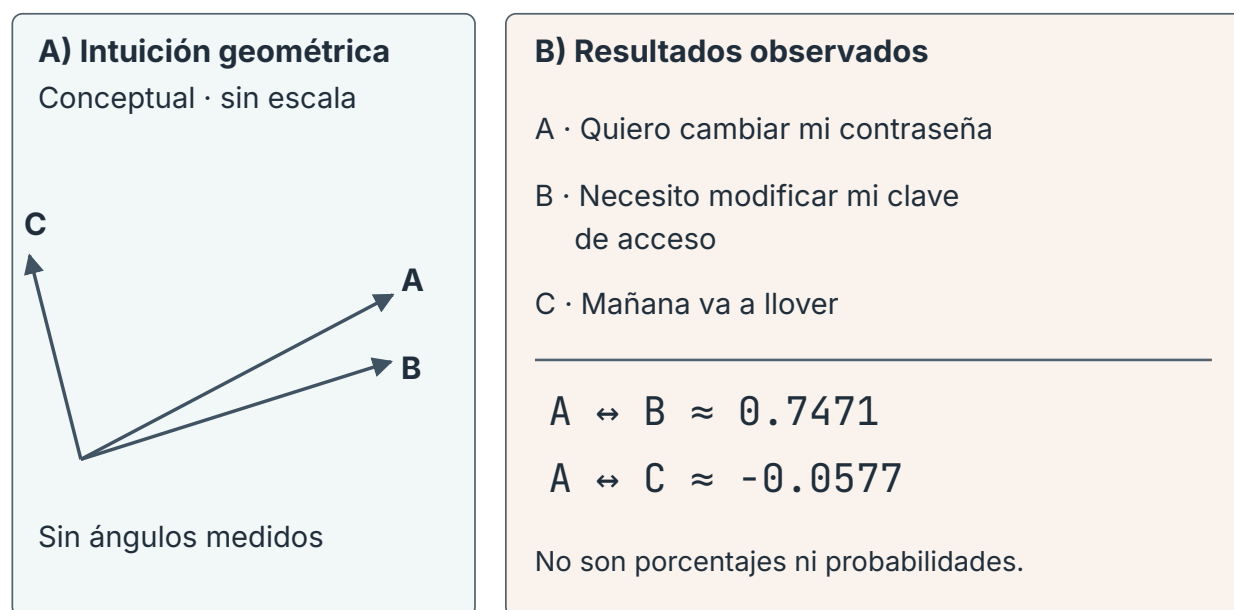


Figura 8 · Dirección de vectores y comparación de las frases A, B y C.

El coseno compara direcciones. Las flechas son conceptuales; las puntuaciones proceden de una ejecución previa en CPU con sentence-transformers==6.1.0 y sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2. No son porcentajes ni probabilidades; un negativo ligero no implica antonimia

Las puntuaciones **no son porcentajes ni probabilidades**. 0.7471 no significa un 74,71 % de certeza ni una cantidad medible de significado compartido. Tampoco hay un valor universal a partir del cual dos frases «signifiquen lo mismo». Los resultados dependen del modelo, de los textos y del procedimiento; los decimales anteriores son referencias de una ejecución, no constantes que deban repetirse exactamente.

Idea clave

Similitud semántica no significa verdad.

Dos textos pueden repetir la misma afirmación equivocada y resultar muy parecidos. Otros pueden tratar sobre el mismo asunto y discrepar en un detalle decisivo. «El pedido ha sido entregado» y «El pedido no ha sido entregado» comparten casi todo su vocabulario y tema, pero la negación cambia lo que afirman. No hemos medido aquí esa pareja: sirve para entender por qué parecido y equivalencia no deben confundirse.

El coseno compara vectores. No consulta los hechos, no comprueba una identidad ni verifica que una solicitud esté autorizada. Una representación útil sigue necesitando interpretación.

16. Qué podemos construir con estas representaciones

La comparación anterior es pequeña, pero contiene una operación reutilizable: representar textos de forma compatible y buscar relaciones entre ellos. Cada aplicación añade sus propios datos, reglas y comprobaciones.

Encontrar información con otras palabras

En una **búsqueda semántica**, representamos una consulta y los textos de una colección, y usamos una comparación para ordenar candidatos. Una consulta sobre «modificar mi clave de acceso» podría recuperar un documento sobre «cambiar la contraseña» aunque la coincidencia literal sea limitada. El salto útil está en relacionar formulaciones diferentes.

Recuperar un candidato cercano no demuestra que responda a la consulta. Puede hablar de restablecer una contraseña olvidada cuando lo que se necesita es cambiar una contraseña conocida. La evaluación debe comprobar si los primeros resultados resuelven la necesidad concreta; a veces conviene combinar señales semánticas con palabras exactas, fechas u otros filtros.

Recomendar por contenido

Podemos representar descripciones de artículos, cursos o productos y buscar contenidos cercanos a uno que interese a una persona. Son **recomendaciones basadas en contenido o afinidad semántica**: usan relaciones entre descripciones, no una lectura directa de los gustos del usuario.

Si alguien consulta un curso sobre lenguaje y datos, otros cursos con temas próximos pueden ser candidatos. La cercanía temática no garantiza nivel adecuado, disponibilidad ni interés personal. Esas condiciones deben añadirse al diseño de la recomendación.

Agrupar y clasificar

Cuando no tenemos categorías preparadas, podemos reunir textos con representaciones próximas. Ese proceso se llama **clustering** o agrupación. Por ejemplo, puede ayudar a explorar los temas recurrentes en mensajes de soporte. Los grupos necesitan interpretación: que un algoritmo los haya formado no significa que sean categorías naturales, completas o igualmente útiles.

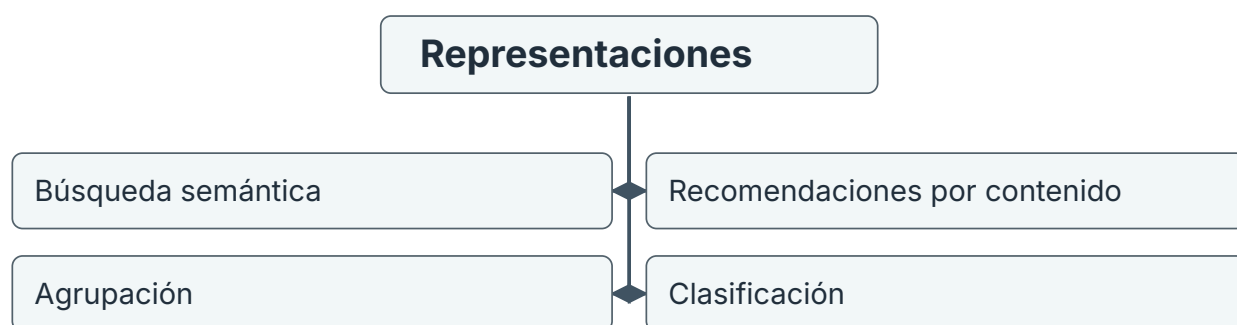
Si ya conocemos las categorías, podemos usar las representaciones como entrada de un **clasificador**, un sistema que aprende a asignar etiquetas a partir de ejemplos etiquetados. Podría distinguir consultas sobre acceso, devoluciones o facturación. El embedding aporta información; la decisión de clasificación requiere un método y una evaluación propios. La puntuación de coseno de nuestra demostración no es, por sí sola, una probabilidad de pertenecer a una categoría.

Recuperar información para RAG

En un sistema **RAG**, siglas de *Retrieval-Augmented Generation* o generación aumentada con recuperación, se conectan dos trabajos diferentes. Primero se **recuperan** fragmentos potencialmente relevantes de una colección. Después, un modelo generativo utiliza el material recuperado como parte de la información disponible para **generar** una respuesta.

Los embeddings pueden ayudar en la recuperación: permiten comparar una consulta con fragmentos almacenados. Un embedding no responde una pregunta por sí mismo; es una representación numérica. El generador es otro componente, aunque ambos formen parte de una misma aplicación.

Si se recupera un fragmento desactualizado o que omite una excepción, la respuesta puede verse afectada. Incluso con documentos adecuados, la generación puede equivocarse. Conviene examinar por separado la calidad de la recuperación y la fidelidad de la respuesta a sus fuentes. Añadir recuperación no convierte automáticamente una respuesta en verdadera.



RAG · dos tareas diferentes

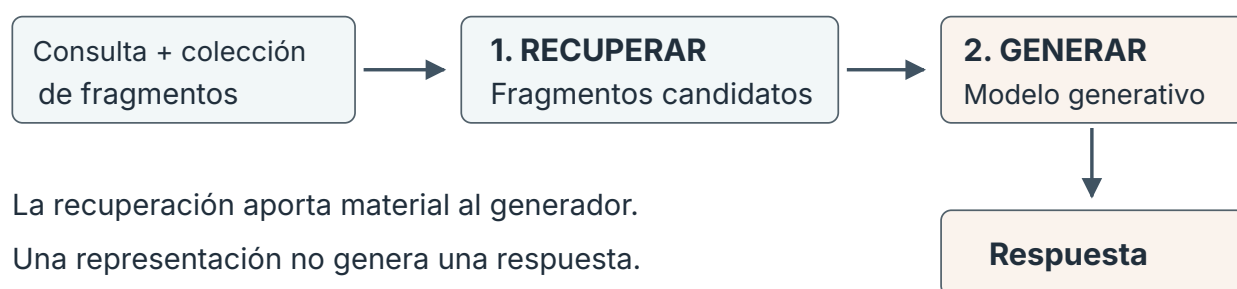


Figura 9 · Aplicaciones de las representaciones y separación entre recuperar y generar.

Las representaciones aportan señales para buscar, organizar y clasificar. En RAG, recuperar candidatos y generar una respuesta son tareas diferentes; ninguna garantiza por sí sola la verdad

17. Cuándo una semejanza puede engañarnos

Las limitaciones no aparecen solo en casos extravagantes. «Banco», sin más contexto, ya es ambiguo. El modelo puede dar una representación, pero eso no significa que la entrada contuviera información suficiente para resolver la duda. Pedir contexto adicional puede ser mejor que forzar una decisión.

También importa el **dominio**, el ámbito de los textos. Una palabra como «alta» puede referirse al registro en un servicio, a una propiedad o a un alta médica. Un modelo útil para mensajes generales puede tratar mal abreviaturas y términos especializados. Ser multilingüe tampoco garantiza el mismo comportamiento en todos los idiomas, variedades o mezclas de lenguas. Hay que comprobar los textos que realmente se van a utilizar.

La elección del modelo afecta a qué relaciones se conservan. Algunos objetivos favorecen la semejanza temática; otros, la detección de formulaciones equivalentes. Dos mensajes sobre una devolución pueden ser cercanos aunque uno la solicite y otro diga que ya se ha realizado. Para dirigir ambos a un departamento quizá baste con el tema; para ejecutar una acción, la diferencia de intención es esencial.

Un **umbral de similitud** es un límite que elegimos para decidir, por ejemplo, si aceptamos una coincidencia. No debe copiarse de esta demostración. Para establecerlo hacen falta ejemplos representativos del problema, incluyendo parejas parecidas que deban rechazarse y parejas con palabras distintas que deban aceptarse. Después se examinan los errores y sus consecuencias.

Si el criterio acepta demasiados candidatos, aparecen **falsas coincidencias**: relaciones que el sistema considera suficientes y la tarea no admite. Si es demasiado exigente, se pierden coincidencias útiles. El equilibrio depende del uso. Mostrar un documento adicional en una búsqueda y ejecutar por error un cambio de cuenta no tienen el mismo coste.

La evaluación debe incluir entradas difíciles, cambios de idioma, negaciones y diferencias pequeñas pero decisivas. También necesita revisar qué ocurre cuando cambian los datos o el modelo. Una demostración con dos comparaciones comprensibles ilustra una capacidad; no mide la fiabilidad de un sistema completo.

18. Sesgos y responsabilidad en las decisiones

Los vectores se aprenden a partir de datos producidos y seleccionados por personas. Esos datos contienen regularidades útiles, pero también ausencias, asociaciones estereotipadas y desigualdades. Si ciertos usos del lenguaje aparecen poco o solo en contextos limitados, sus representaciones pueden resultar menos útiles o arrastrar asociaciones inadecuadas.

Imagina un sistema que organiza solicitudes de ayuda. Si se ha evaluado principalmente con mensajes escritos en una variedad de español, podría reconocer peor peticiones expresadas de otra manera. Un sistema posterior que priorice atención a partir de esas representaciones

podría convertir un error de comparación en una diferencia de trato. Es un riesgo que debe comprobarse; no una medición que hayamos realizado con el modelo de esta práctica.

La apariencia numérica no vuelve neutral una decisión. Elegir qué textos entran, qué similitud se usa, qué umbral se aplica y qué ocurre después son decisiones humanas. Automatizar el cálculo no elimina la responsabilidad sobre sus consecuencias.

En decisiones sensibles, como acceso a servicios o evaluación de personas, la similitud semántica no debe ser el único criterio sin evaluación, controles y supervisión apropiados. Evaluar incluye comprobar errores con poblaciones y formas de expresión relevantes. Supervisar requiere que una persona pueda entender los límites, revisar el caso y corregir una decisión, no solo confirmar automáticamente una puntuación.

La respuesta útil no es rechazar toda automatización, sino definir para qué sirve, dónde puede fallar y qué protección necesita quien soporta el error. En una búsqueda puede bastar con ofrecer alternativas y permitir contrastarlas; en una decisión con consecuencias importantes harán falta garantías adicionales.

19. Privacidad: dónde se procesa el texto importa

La práctica descrita utiliza un **modelo público** descargable y realiza la **inferencia**, es decir, el cálculo con un modelo ya entrenado, en el entorno de ejecución o *runtime* empleado. Las referencias se obtuvieron en CPU en Google Colab, sin recurrir a una API comercial de embeddings. Descargar un modelo y utilizar un servicio remoto que recibe textos para devolver vectores son modos de trabajo diferentes.

Eso no convierte cualquier entorno en privado por definición. Si el runtime está alojado en un servicio, no es lo mismo que ejecutar en el propio ordenador. Tampoco que un modelo sea público determina por sí solo qué ocurre con los textos introducidos, los archivos guardados o el acceso a los resultados.

Antes de enviar información sensible en un proyecto real, hay que entender dónde se procesa, qué servicio interviene, si se almacena, quién tiene permisos y qué política resulta aplicable. La revisión debe corresponder al entorno concreto; no podemos deducirla del nombre del modelo ni hacer una afirmación general sobre todos los servicios.

Los embeddings tampoco deben tratarse automáticamente como datos anónimos: son representaciones derivadas de los textos y pueden conservar información útil sobre ellos. Minimizar datos innecesarios, restringir accesos y usar ejemplos ficticios cuando basten son decisiones de diseño que reducen exposición. Para aprender los conceptos de este documento no hacen falta nombres reales, contraseñas ni otros datos personales.

20. Coste y recursos: una representación también ocupa espacio

Un modelo ya entrenado evita que tengamos que entrenarlo para cada uso, pero no elimina el coste de utilizarlo. Primero hay que disponer de sus archivos, descargarlos cuando corresponda y cargarlos en memoria. Después, cada entrada exige procesamiento en CPU o GPU y tarda un tiempo en producir su resultado. Esa espera es la **latencia**.

Guardar embeddings también consume almacenamiento. Tres vectores son fáciles de manejar, pero una colección grande requiere mantener muchas representaciones y buscar entre ellas. Se pueden preparar estructuras de búsqueda, llamadas índices, para acelerar la consulta. Eso añade decisiones sobre memoria, actualización y precisión de la recuperación. No es la misma tarea comparar dos frases que mantener una colección cambiante de millones de fragmentos.

Cambiar de modelo puede obligar a volver a representar la colección, porque sus vectores no tienen por qué ser compatibles con los anteriores. Dividir documentos en más fragmentos puede conservar detalles, pero aumenta el número de representaciones que hay que calcular y almacenar. Las decisiones de calidad y coste están conectadas.

Por eso el modelo «mejor» no es necesariamente el más grande. Debe responder al problema, al idioma, a la calidad necesaria, al presupuesto y a la latencia aceptable. Un método sencillo de coincidencia léxica puede ser suficiente en una tarea acotada; otro caso puede justificar un modelo de embeddings. La comparación debe hacerse sobre necesidades y datos reales, sin atribuir superioridad automática a una técnica por ser más reciente.

Condiciones de uso · esquema conceptual

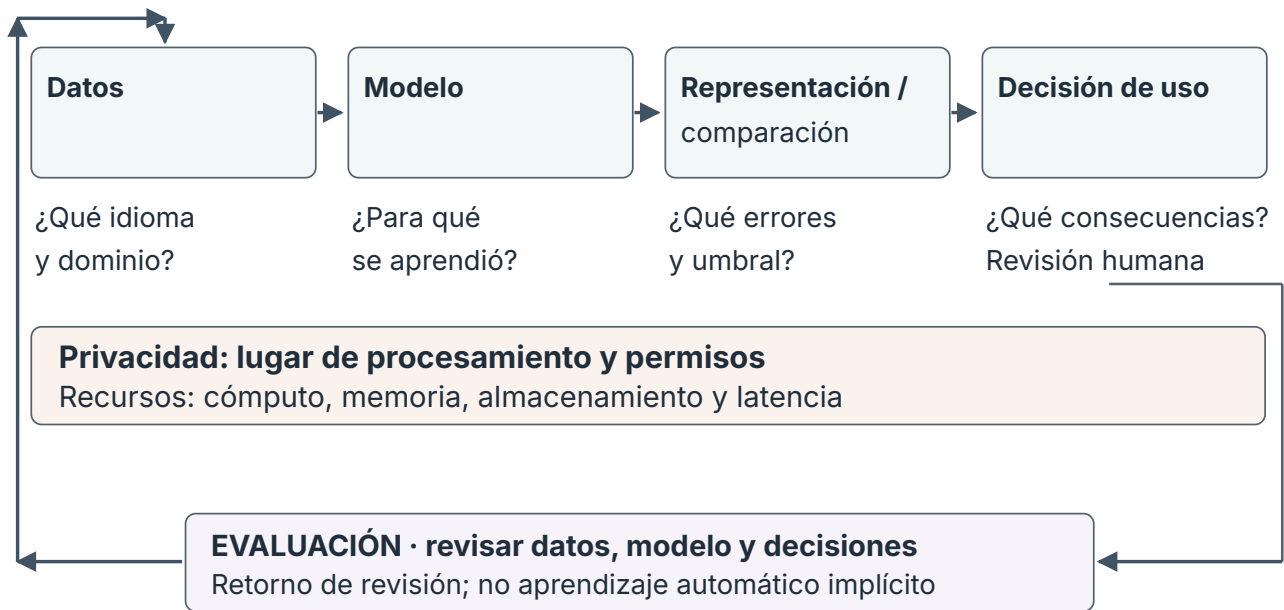


Figura 10 · Modelo, datos y decisiones humanas condicionan el uso del resultado.

La utilidad depende de datos, modelo y decisiones de uso. Evaluar errores, proteger información y prever revisión humana son partes del diseño, no propiedades garantizadas por una puntuación

21. Reto de repaso: cambiar los textos y conservar el razonamiento

Propuesta

Considera ahora estas tres frases:

A = "Quiero devolver el producto que compré"
B = "Necesito hacer una devolución de mi compra"
C = "¿Qué tiempo hará este fin de semana?"

Antes de consultar la interpretación, razona qué comparación esperas que sea mayor: A ↔ B o A ↔ C. Explica qué relación lingüística sostiene tu expectativa y qué no permitiría concluir una puntuación alta. Piensa también qué forma tendría la colección de vectores si mantuviéramos el mismo modelo de 384 valores por frase.

Puedes resolver estas preguntas leyendo los textos. Si decides llevarlas al notebook, el recorrido es el mismo: representar las tres frases con el modelo indicado, calcular ambas similitudes del coseno y comparar los resultados. No hace falta aprender otra técnica.

Solución e interpretación

A solicita devolver un producto comprado. B formula una petición muy parecida mediante «hacer una devolución» y «mi compra». C plantea una pregunta meteorológica. Por eso esperamos mayor similitud para $A \leftrightarrow B$ que para $A \leftrightarrow C$.

En la ejecución previa del reto en CPU, con `sentence-transformers==6.1.0` y `sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2`, se observaron estas referencias:

```
Forma: torch.Size([3, 384])  
A ↔ B ≈ 0.7669  
A ↔ C ≈ -0.0830
```

Cambiar los textos no cambia la dimensión del modelo: seguimos teniendo tres frases y 384 valores para cada una. La diferencia entre las comparaciones encaja con la expectativa, y el pequeño negativo de $A \leftrightarrow C$ no convierte las frases en antónimas.

El resultado apoya una relación útil entre formulaciones distintas. No demuestra que la devolución esté permitida, que la compra exista o que quien escribe tenga derecho a realizarla. Para resolver esas cuestiones harían falta datos y verificaciones adicionales. Esa separación entre semejanza de una petición y validez de una acción es parte de interpretar correctamente el ejercicio.

22. Volver a la pregunta inicial

¿Cómo puede un ordenador trabajar con lenguaje? Construyendo representaciones con las que realizar operaciones útiles. Primero identificamos piezas; después podemos contar sus apariciones, ponderarlas o utilizar vectores aprendidos. El contexto permite transformar representaciones según la secuencia. Un modelo preparado para frases puede reunir información en vectores comparables, y una medida como el coseno ofrece una señal para buscar, organizar o apoyar otras tareas.

Nada de ese recorrido exige afirmar que el ordenador entiende como una persona. Sí permite explicar qué se calcula, qué información se conserva y dónde hace falta comprobar la interpretación.

Tres ideas sostienen el conjunto:

1. **Un ID identifica; un embedding representa relaciones aprendidas.**
2. **El contexto cambia la representación y la interpretación.**
3. **Una similitud útil sigue necesitando evaluación, contexto y criterio humano.**

Si te apetece seguir explorando, vuelve al notebook y prueba frases propias: dos peticiones equivalentes, una negación o un término de otro ámbito. Anticipar el resultado y contrastarlo con lo que produce el modelo puede enseñar tanto como una comparación que sale según lo esperado.

Glosario

NLP / PLN. Procesamiento del lenguaje natural: campo que desarrolla métodos computacionales para trabajar con lenguaje. NLP corresponde a sus siglas en inglés.

Token. Unidad de texto que utiliza un tokenizador. Puede ser una palabra, parte de ella o un signo, entre otras posibilidades.

Tokenizador. Herramienta que prepara las unidades de una entrada según un vocabulario y unas reglas. En el procesamiento para un modelo también permite obtener sus identificadores.

Vocabulario. Repertorio de entradas reconocidas por un sistema. En un tokenizador, relaciona piezas con IDs.

ID. Identificador de una entrada. Su valor numérico no expresa por sí mismo significado ni distancia semántica.

Vector. Lista ordenada de números. Según cómo se construya, puede representar casillas activas, conteos o información aprendida.

Embedding. Representación vectorial aprendida que puede conservar relaciones útiles para una tarea. No es una definición completa ni objetiva del significado.

Subword. Pieza reutilizable de texto que puede coincidir con una palabra o con parte de ella. No tiene por qué ser una unidad lingüística con significado propio.

BPE. Método de construcción de vocabulario basado en fusionar pares vecinos frecuentes. Al tokenizar textos nuevos se aplican las reglas aprendidas.

WordPiece. Método de tokenización en piezas cuyo criterio de construcción difiere del de BPE. No basta con contar el par más frecuente para describir su selección.

Unigram. Método que parte de un repertorio amplio de piezas candidatas y elimina las menos útiles. Emplea probabilidades para valorar segmentaciones.

Contexto. Información que rodea una aparición del texto y puede ayudar a interpretarla. En el modelo depende de qué entrada y posiciones tenga disponibles.

Atención. Mecanismo que combina información de distintas posiciones con pesos calculados por el modelo. No implica atención consciente.

Transformer. Arquitectura que hace central la atención para procesar relaciones entre posiciones de una secuencia. Sus representaciones también necesitan información sobre el orden.

BERT. Ejemplo de Transformer encoder que produce representaciones contextuales utilizando información a ambos lados de una posición. No equivale por sí solo a un modelo preparado para comparar frases mediante un único vector.

Sentence embedding. Vector que representa una frase o secuencia para tareas como la comparación semántica. Su utilidad depende del modelo, el entrenamiento y la forma de obtenerlo.

Similitud del coseno. Medida que compara la dirección de dos vectores no nulos. En embeddings ofrece una señal de relación, no una probabilidad ni una prueba de verdad.

RAG. Generación aumentada con recuperación: combina búsqueda de información con generación de una respuesta. Recuperar y generar son operaciones distintas.

Autoevaluación

Preguntas

1. Si intercambiamos los IDs de perro y avión y actualizamos el catálogo, ¿cambia el significado de las palabras? ¿Por qué?
2. ¿Qué evita one-hot y qué relación sigue sin mostrar entre perro, gato y avión?
3. ¿Por qué «perro persigue gato» y «gato persigue perro» pueden ser indistinguibles para Bag of Words?
4. ¿Por qué una palabra frecuente en un documento puede recibir poca importancia con TF-IDF?
5. ¿Qué problema presenta un único vector fijo de banco? ¿Lo resuelve cambiar su ID?
6. ¿Por qué dividir perritos en piezas conocidas no garantiza que el modelo interprete bien la palabra? ¿Reaprende BPE su vocabulario al recibirla?
7. ¿Cómo puede aparecer _banco con el mismo ID en dos frases y terminar con representaciones contextuales diferentes?
8. ¿Qué significa `torch.Size([3, 384])`? ¿Serviría cualquier salida de cualquier Transformer para comparar esas frases?
9. ¿Qué podemos inferir de que $A \leftrightarrow B$ tenga mayor coseno que $A \leftrightarrow C$? ¿Qué no demuestran ni el valor alto ni un negativo ligero?
10. En RAG, ¿qué aporta la recuperación y qué añade la generación? ¿Puede un embedding responder por sí solo?

- L1. Antes de usar una similitud para decidir sobre personas, ¿qué revisarías además de la puntuación? ¿Copiarías un umbral de este ejemplo?
- L2. ¿Por qué utilizar un modelo público no resuelve automáticamente las preguntas de privacidad y coste?
-

Respuestas orientativas

1. No cambia. El ID sirve para localizar la entrada; su magnitud y el orden del catálogo no definen el significado.
2. Evita tratar el identificador como una magnitud ordenada. Sigue dando posiciones independientes, sin expresar mayor relación entre perro y gato.
3. Contienen las mismas palabras con las mismas frecuencias. BoW descarta el orden y, en este ejemplo, quién realiza la acción.
4. Si aparece en casi toda la colección, ayuda poco a distinguir ese documento. TF-IDF combina presencia local con rareza entre documentos.
5. Mezcla señales de usos diferentes sin adaptar el vector a cada aparición. Cambiar un identificador no aporta el procesamiento del contexto que falta.
6. Segmentar permite preparar una entrada; interpretar requiere representaciones y procesamiento aprendidos. BPE aplica reglas previas, no aprende de nuevo con cada frase.
7. El ID consulta la misma entrada del vocabulario, pero el procesamiento posterior incorpora posiciones y otras piezas distintas en cada frase.
8. Hay tres vectores de 384 valores, uno por frase para este modelo. Tener representaciones contextuales no garantiza un vector de frase útil para comparar: importa cómo se obtiene y para qué se entrenó.
9. El modelo sitúa A y B en direcciones más próximas que A y C. Un valor alto no prueba equivalencia exacta, verdad ni autorización; un pequeño negativo no prueba antonimia.
10. La recuperación selecciona material candidato; la generación elabora una respuesta con información disponible. El embedding ayuda a comparar y recuperar, pero no es una respuesta.
- L1. Revisaría datos, idiomas y grupos relevantes, errores, consecuencias, posibles sesgos, controles y supervisión. El umbral necesita validación con el problema real.
- L2. Hay que conocer dónde se procesa y almacena la información y quién accede a ella. Descargar, cargar, calcular, almacenar vectores y buscar entre ellos también consume recursos.